

JEONME

Dokumentasi Docker & Infrastruktur

Dockerfile, Docker Compose, Nginx & TLS — Local, Staging, Production

Dokumen	INFRA-DOC v1.0
Status	Berfungsi, sudah diaudit
Tanggal	3 Juli 2026
Stack	Docker + Docker Compose · Nginx · Let's Encrypt (Certbot)
Lokasi kode	<code>docker/</code> , <code>docker-compose*.yaml</code>

1. Prinsip Strategi Container

Image Docker dibangun **sekali** di CI (job `build-images` pada `ci.yml`), lalu image yang **sama persis** dipakai di staging maupun production — bukan build ulang di server. Ini menjamin apa yang diuji di CI adalah apa yang sungguhan berjalan di production (lihat CICD-GUIDE Bagian 1).

2. Dockerfile API (`docker/api/Dockerfile`)

Multi-stage: `builder` (`golang:1.25-alpine`) mengompilasi binary statis (`CGO_ENABLED=0`), lalu `runner` (`alpine:3.20`) hanya menyalin binary jadi + folder `migrations` . Hasil akhir: image sangat kecil (~15–20MB) tanpa toolchain Go sama sekali di image final.

STAGE	BASE IMAGE	ISI
<code>builder</code>	<code>golang:1.25-alpine</code>	<code>go mod download</code> → <code>go build -ldflags="-w -s"</code> (strip simbol debug, image lebih kecil)
<code>runner</code>	<code>alpine:3.20</code>	Binary <code>api</code> + folder <code>migrations</code> (dipakai subcommand <code>migrate</code> saat deploy) + <code>ca-certificates</code> , <code>tzdata</code>

Expose port `8080` , entryptoint langsung `./api` (tanpa argumen = jalankan server; dengan argumen `migrate ...` = jalankan migrasi lalu keluar, lihat Dokumentasi Backend Bagian 8).

3. Dockerfile Web (`docker/web/Dockerfile`)

Tiga stage: `deps` (install dependency), `builder` (`next build` , menghasilkan `.next/standalone`), `runner` (hanya menyalin hasil standalone + static + public — **tanpa** `node_modules` penuh).

STAGE	BASE IMAGE	ISI
<code>deps</code>	<code>node:24-alpine</code>	<code>npm ci</code> (instal reproducible dari <code>package-lock.json</code>)
<code>builder</code>	<code>node:24-alpine</code>	<code>next build</code> — butuh <code>output: 'standalone'</code> di <code>next.config.js</code>
<code>runner</code>	<code>node:24-alpine</code>	<code>public/</code> + <code>.next/standalone</code> + <code>.next/static</code> , CMD <code>["node", "server.js"]</code>

Node dinaikkan dari versi 20 ke 24 (Active LTS) di kedua Dockerfile — Node 20 sudah EOL per 30 April 2026 (lihat README).

4. Docker Compose — Local Dev (`docker-compose.yml`)

SERVICE	IMAGE/BUILD	PORT	CATATAN
---------	-------------	------	---------

web	build dari docker/web/Dockerfile	3000	NEXT_PUBLIC_API_BASE_URL=http://localhost:8080/api/v1
api	build dari docker/api/Dockerfile	8080	depends_on : db (healthy), redis (started)
db	postgres:16-alpine	5432	Healthcheck pg_isready ; migrasi tidak auto-run — jalankan manual make migrate-up
redis	redis:7-alpine	6379	—

5. Docker Compose — Staging (docker-compose.staging.yml)

Image di-pull dari GHCR: `ghcr.io/${GHCR_REPO}/${web,api}:${IMAGE_TAG:-latest}`. `GHCR_REPO` harus diisi persis `owner/repo` di `.env` VPS (menggantikan placeholder `OWNER` yang ditemukan saat audit CI/CD). Nginx staging memakai konfigurasi terpisah di `docker/nginx/conf.d/staging/` (HTTP saja di port `8081`) — lihat Bagian 7.

6. Docker Compose — Production (docker-compose.prod.yml)

SERVICE	CATATAN
web / api	Sama seperti staging, tag image dikontrol <code>IMAGE_TAG</code> tanpa fallback (harus selalu diisi valid oleh workflow deploy)
worker (dikomentari)	TODO: <code>main.go</code> belum punya subcommand <code>worker</code> — perlu job queue (mis. <code>asynq</code> berbasis Redis) untuk proses async sebelum diaktifkan
db	<code>postgres:16</code> (bukan alpine — image resmi penuh untuk production)
redis	<code>redis:7-alpine</code>
nginx	Port 80 & 443, memuat <code>docker/nginx/conf.d/production/</code> + volume Certbot
certbot	Image <code>certbot/certbot</code> , <code>entrypoint: "true"</code> (tidak auto-start) — dipakai sekali untuk penerbitan sertifikat awal dan rutin lewat cron untuk renewal

7. Nginx & TLS

`docker/nginx/nginx.conf` — konfigurasi dasar (`gzip`, `client_max_body_size 20m`, `include conf.d/*.conf`), sama untuk semua environment. Konfigurasi domain dipisah per environment sejak audit CI/CD (sebelumnya satu file dipakai bersama, berisiko konflik kebutuhan TLS):

FILE	ENVIRONMENT	TLS	CATATAN
<code>conf.d/production/jeonme.conf</code>	Production	Ya — server block 443 lengkap	Server block 80 khusus redirect + ACME challenge /.well-known/acme-challenge/ ; <code>ssl_protocols TLSv1.2 TLSv1.3</code> (REQ-NF-03)

<code>conf.d/staging/jeonme.conf</code>	Staging	Tidak (di level Nginx ini)	Asumsi Cloudflare di depan yang menerminasi TLS untuk <code>staging.jeonme.com</code> (lihat Technical Design Document §7.1)
---	---------	----------------------------	--

Kedua file melakukan reverse proxy yang sama: `/api/` → `api:8080` , `/` → `web:3000` , dengan header `X-Real-IP` / `X-Forwarded-For` / `X-Forwarded-Proto` diteruskan.

Setup sertifikat awal untuk production terdokumentasi lengkap sebagai komentar di dalam `conf.d/production/jeonme.conf` itu sendiri, dan diotomasi lewat `scripts/issue-certbot-cert.sh` (lihat Dokumentasi CI/CD & Deployment).

8. Environment Variables — Rujukan Lengkap

FILE	DIPAKAI OLEH	VARIABEL KUNCI
<code>.env.example</code> (root)	VPS staging/production, dibaca <code>docker compose</code>	<code>GHCR_REPO</code> , <code>IMAGE_TAG</code> , <code>DATABASE_URL</code> , <code>POSTGRES_PASSWORD</code> , <code>JWT_SECRET</code> , <code>XENDIT_*</code> , <code>CORS_ALLOWED_ORIGINS</code> , <code>NEXT_PUBLIC_API_BASE_URL</code>
<code>apps/api/.env.example</code>	Development backend lokal (tanpa Docker)	<code>APP_ENV</code> , <code>APP_PORT</code> , <code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>JWT_SECRET</code> , <code>XENDIT_*</code> , <code>CORS_ALLOWED_ORIGINS</code>
<code>apps/web/.env.example</code>	Development frontend lokal	<code>NEXT_PUBLIC_API_BASE_URL</code>

Ganti seluruh nilai `CHANGE_ME` sebelum production, dan jangan pernah commit file `.env` asli (sudah masuk `.gitignore`).

9. Volumes & Persistensi Data

VOLUME	DIPAKAI DI	ISI
<code>pgdata</code>	local dev & production	Data PostgreSQL
<code>pgdata_staging</code>	staging	Data PostgreSQL staging (terpisah dari production)
<code>./certbot/conf</code> , <code>./certbot/www</code>	production	Sertifikat Let's Encrypt & webroot ACME challenge — bind mount, bukan named volume, supaya mudah di-backup manual

10. Catatan Versi & Keamanan

Ringkasan dari README — perubahan versi yang ditemukan & diperbaiki saat penyusunan boilerplate:

- **Next.js 14.2.5 → 16.2.10 LTS** — versi 14 sudah *end-of-life* sejak Oktober 2025, rentan CVE kritis App Router (RCE "React2Shell", CVE-2025-66478). React ikut naik ke 19.2.7.
- **Node.js 20 → 24 (Active LTS)** di Dockerfile & CI — Node 20 EOL 30 April 2026.

- **Go 1.22** → **1.25** di `go.mod` , Dockerfile, CI — Go hanya mendukung dua rilis mayor terbaru.
- Masih ada 1 kerentanan moderate (XSS PostCSS) dari dependency internal Next.js sendiri, bukan dependency langsung proyek — pantau lewat `npm audit` (sudah otomatis berjalan non-blocking di CI, lihat Dokumentasi CI/CD).